

哈希

为什么要有哈希？

假设我们要设计一个系统来存储将员工手机号作为主键的员工记录，并希望高效地执行以下操作：

- 1、插入电话号码和相应的信息。（插入）
- 2、搜索电话号码并获取信息。（查找）
- 2、删除电话号码及相关信息。（删除）

我们可以考虑使用以下数据结构来维护不同电话所对应的信息：

- 1、数组
- 2、链表
- 3、平衡二叉树（红黑树等等）
- 4、直接访问表

对于数组和链表而言，我们需要以线性的方式进行查找，这在实际应用中代价太大；如果我们使用数组且保证数组中电话号码有序排列，那么使用二分查找可将查找电话号码的时间复杂度降低，但同时由于要维持数组的有序性，插入和删除操作的代价将变大。

对于平衡二叉查找树而言，插入、查找和删除操作的时间复杂度相同，这似乎已经看着很不错了，那么是否有更好的数据结构呢？

下标（电话号码）	指向记录的地址
15002516299	0x0001
10000000000	0x0123
15000001234	0xffff
18845678954	Null
.....	

再来看直接访问表的方式，首先创建一个大数据组（至少能够用电话号码作为数组下标），如果电话号码没有出现在数组当中，就在相应下标填充 `NULL`；如果电话号码出现了，则下标中数据填充该电话号码关联的记录地址。

这样一来，插入、查找和删除的时间复杂度将降为 $O(1)$ ，比如插入一条记录（15000000000，`0xFF0A` “可爱的读者”），只需要将手机号码 15000000000 当做下标，然后在该下标的位置填充记录的地址，即 `arr[15000000000] = 0xFF0A;`

但是直接访问表有实践上的限制。首先需要申请额外的存储空间，并且存在大量的空间浪费。比如对于一个含有 n 位的电话号码，我们需要一个比较大的空间复杂度，其中 m 表示数据本身所占用的空间。另外一个问题是，编程语言本身提供的整型无法表示电话号码。

哈希是对直接访问表的改进。使用哈希函数将给定的电话号码或任何其他键转换为较小的数字，将该较小的数字称为哈希表的索引（哈希值）。

什么是哈希表？

哈希表和直接访问表很类似，同样是一个用于存储指向给定电话号码对应记录的指针的数组，只不过，此时的数组下标不再是电话号码，而是经过哈希函数映射后的输出值。

什么是哈希函数？

哈希函数用于将一个大数（手机号码）或字符串映射为一个可以作为哈希表索引的较小整数的函数。比如活动开发中经常使用的 MD5 和 SHA 都是历史悠久的 Hash 算法。

```
[lovefxs@localhost ~]$ echo -n "I love J" | openssl md5
(stdin)= ef821d9b424fd5f0aac7faf029152e04
```

一个好的哈希函数应该满足四个条件：

1、执行效率要高（效率高）

对于一段很长的字符串或者二进制文本也能快速计算出哈希值。

2、散列结果应当具有同一性（输出值尽量均匀，越均匀冲突就越少）。（同一性）

例如对于电话号码而言，一个好的哈希函数应该考虑电话号码的后四位，而一个糟糕的哈希算法可能考虑电话号码的前四位，因为后四位更有区分性，相当于输入更加分散，那么输出也可能更加均匀，当然这两种选择方式都不是什么好办法，只是希望大家理解同一性原理。

3、雪崩效应（微小的数值变化使得输出值发生巨大的变化）

```
[lovefxs@localhost ~]$ echo -n "I love J" | openssl md5
(stdin)= ef821d9b424fd5f0aac7faf029152e04
[lovefxs@localhost ~]$ echo -n "I love Y" | openssl md5
(stdin)= fb49af24bebae07658650ae8eb1c0f5b
```

其中输入 `I love J` 和 `I love Y` 只改变了一个字母，输出值却千差万别。

4、从哈希函数的输出值不可反向推导出原始的数据。（不可反向推导）

比如上面的原始数据 `I love J` 与经过 MD5 算法映射后的输出值之间没有对应关系。

什么是 Hash 碰撞？

由于哈希函数的原理是将输入空间的一个较大的值映射成 hash 空间内一个较小的值，那么就会出现两个不同的输入值被映射到了同一个较小的输出值。当一个新插入的值被哈希函数映射到了哈希表中一个已经被占用的槽，就认为产生了 Hash 碰撞（冲突）。

那么这种冲突是否可以避免呢？

答案是只能缓解，不可避免。

由于哈希函数的原理是将输入空间一个较大的值映射到一个较小的 Hash 空间内，而 Hash 空间一般远小于输入的空间。根据抽屉原理，一定会存在不同的输入被映射成同一输出的情况。

何为抽屉原理？



比如桌上有十个苹果，要把这十个苹果放到九个抽屉里，无论怎样放，我们会发现至少会有一个抽屉里面放不少于两个苹果。这一现象就是“抽屉原理”。抽屉原理的一般含义为：“如果每个抽屉代表一个集合，每一个苹果就可以代表一个元素，假如有 $n+1$ 个元素放到 n 个集合中去，其中必定有一个集合里至少有两个元素。” 抽屉原理有时也被称为鸽巢原理。它是组合数学中一个重要的原理。

知道了为什么哈希碰撞不可避免，那哈希碰撞该如何缓解呢？

Hash 碰撞的解决方案？

链地址法

链地址法的思想就是将所有发生碰撞的元素用一个单链表串起来。

我们以一个简单的哈希函数 $H(key) = key \text{ MOD } 7$ （除数取余法）对一组元素 [50, 700, 76, 85, 92, 73, 101] 进行映射，来理解链地址法处理 Hash 碰撞。

除数为 7，初始化一个大小为 7 的空 Hash 表：

0	
1	
2	
3	
4	
5	
6	

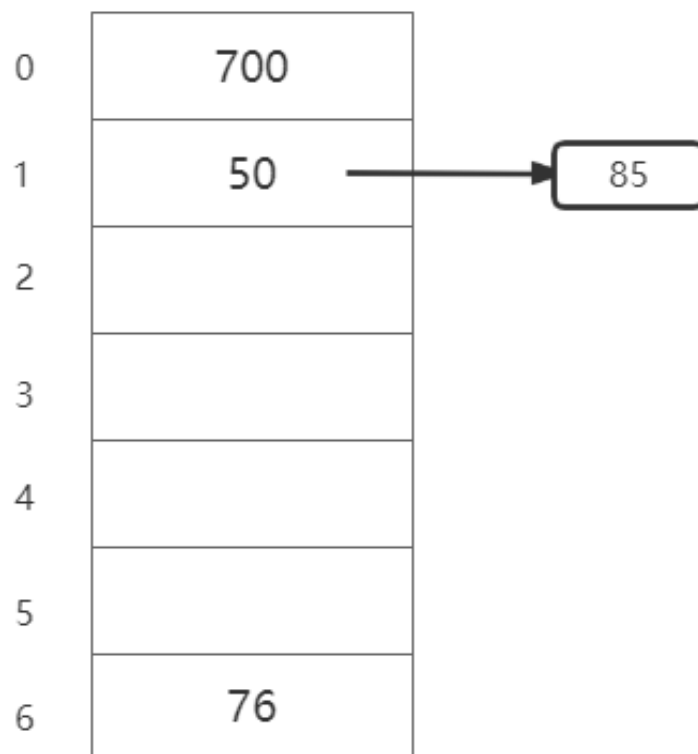
然后插入元素 **50**，首先对 $50 \% 7 = 1$ ，得到其哈希值 **1**，在下标为 **1** 的位置插入 50：

0	
1	50
2	
3	
4	
5	
6	

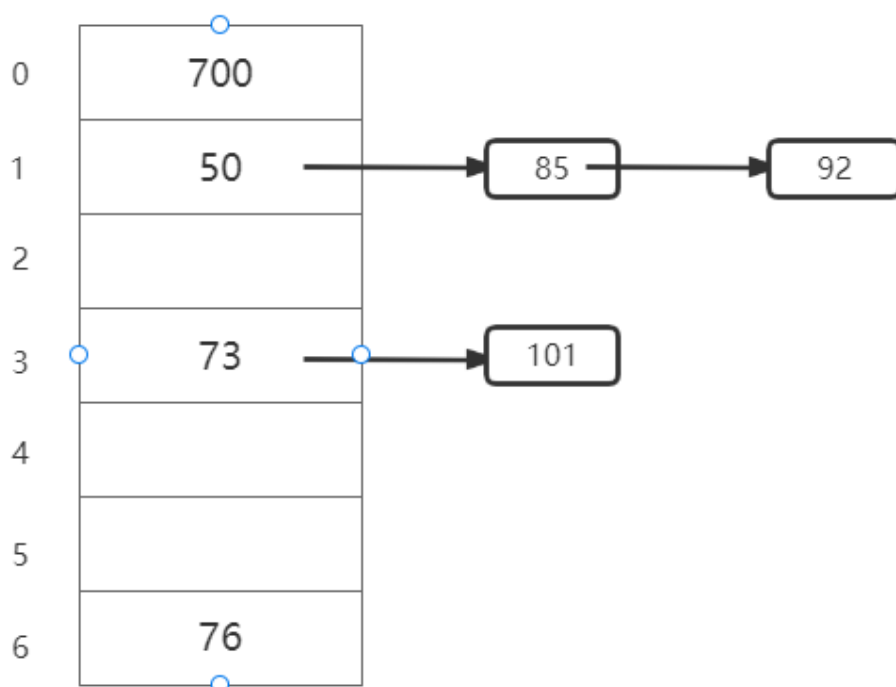
然后计算 $700 \% 7 = 0$ ， $76 \% 7 = 6$ ，得到的哈希值均未发生碰撞，填入相应位置：

0	700
1	50
2	
3	
4	
5	
6	76

然后计算 $85 \% 7 = 1$ ，但是下标为 **1** 的位置已经有元素 **50**，发生了碰撞，所以使用单链表将 **85** 和 **50** 链接起来：



以同样的方式插入所有元素得到如下的哈希表。链地址法解决冲突的方式与图的邻接表存储方式在样式上很相似，思想还是蛮简单，发生冲突，就用单链表组织起来。



链地址法的优缺点：

优点：

- 1、实现起来简单；
- 2、哈希表永远不会溢出，我们可以向单链表中添加更多的元素；
- 3、对于哈希函数和装填因子（后面会说）的选择没啥要求；
- 4、在不知道要插入和删除关键字多少和频率的情况下，链地址法有绝对优势。

缺点：

- 1、由于使用链表来存储关键字，链地址法的缓存性能不佳。开放寻址法使用连续的地址空间进行存储，所以缓存性能更好。
- 2、浪费空间（因为哈希表的有些位置从未被使用）。
- 3、如果链表太长，在最坏的情况下查找的时间复杂度将变为 $O(n)$ 。
- 4、需要额外的空间存储链表指针。

链地址法的性能

假设任何一个关键字都以相等的概率被映射到哈希表的任意一个槽位。其中 m 表示哈希表的槽位数， n 表示插入到哈希表中的关键字的数目。装填因子（load factor）为 n/m （将 n 个球随机均匀地扔进 m 个箱子里的概率）。期望的查找，插入和删除的时间均等于 $O(1 + n/m)$ 当 n/m 为 $O(1)$ 量级时，链地址法的插入、查找和删除操作的时间复杂度就是 $O(1)$ 量级。极端情况下我们将 n 个数都扔进了同一个槽，也就是 n 个数形成了一个单链表，那么时间复杂度将降为 $O(n)$ ，但这个概率微乎其微。

其微。

开放地址法

与链地址法一样，开放地址法也是一个经典的处理冲突的方式。只不过，对于开放地址法，所有的元素都是存储在哈希表当中的，所以无论任何时候都要保证哈希表的槽位数 m 大于或等于关键字的数目 n （必要的时候，我们还会复制旧数据，然后对哈希表进行动态扩容）。开放地址法分三种方式。

线性探测法

设 $\text{Hash}(\text{key})$ 表示关键字 key 的哈希值， M 表示哈希表的槽位数（哈希表的大小）。

线性探测法则可以表示为：

如果 $\text{Hash}(x) \% M$ 已经有数据，则尝试 $(\text{Hash}(x) + 1) \% M$ ；

如果 $(\text{Hash}(x) + 1) \% M$ 也有数据了，则尝试 $(\text{Hash}(x) + 2) \% M$ ；

如果 $(\text{Hash}(x) + 2) \% M$ 也有数据了，则尝试 $(\text{Hash}(x) + 3) \% M$ ；

我们同样以哈希函数 $H(\text{key}) = \text{key} \bmod 7$ （除数取余法）对 $[50, 700, 76, 85, 92, 73, 101]$ 进行映射，来理解线性探测法处理 Hash 碰撞。

依次计算 $50 \% 7 = 1$ 、 $700 \% 7 = 0$ 及 $76 \% 7 = 6$ ，均没有发生冲突（碰撞），则直接放入相应的位置：

0	700
1	50
2	
3	
4	
5	
6	76

计算 $85 \% 7 = 1$ ，但是下标 1 的位置已经有元素 **50**，发生碰撞，则寻找下一个可以存放元素 **85** 的空位 $(85 \% 7 + 1) \% 7 = 2$ ，下标二没有元素，所以将 85 放进去：

0	700
1	50
2	85
3	
4	
5	
6	76

计算 $92 \% 7 = 1$ ，**50** 已经在 1 的位置，发生碰撞；线性探测下一个位置 $(92 \% 7 + 1) \% 7 = 2$ ，**85** 已经在 2 的位置，发生碰撞；线性探测下一个位置 $(92 \% 7 + 2) \% 7 = 3$ ，此时为空位，填入 **92**：

0	700
1	50
2	85
3	92
4	
5	
6	76

计算 $73 \% 7 = 3$ ，92 已经占用了 3 号位置，发生碰撞；线性探测下一个位置 $(73 \% 7 + 1) \% 7 = 4$ ，此时为空位，填入 73：

0	700
1	50
2	85
3	92
4	73
5	
6	76

计算 $101 \% 7 = 3$, 3 号位置已经有元素, 发生碰撞; 线性探测下一个位置 $(101 \% 7 + 1) \% 7 = 4$, 4 号位置已有元素, 发生碰撞; 线性探测下一个位置 $(101 \% 7 + 2) \% 7 = 5$, 此时为空, 填入 101 :

0	700
1	50
2	85
3	92
4	73
5	101
6	76

到这里你不难看出线性探测的缺陷, 容易产生聚集 (发生碰撞的元素形成组, 比如 **[50,85,92,73,101]** 之间均是由于碰撞而紧挨着), 而且发生碰撞的情况大大增加, 需要花费更多的时间来寻找空闲的槽位和查找元素。

至于 (lazy delete) 懒删除就是并不将存储空间释放, 只是将里面的数据清除。

比如删除元素 **92** , 先计算 $92 \% 7 = 1$, 但是发现下标 1 是 50, 线性向下探测 $(92 \% 7 + 1) \% 7 = 2$, 下标 2 为 85, 继续线性向下探测 $(92 \% 7 + 2) \% 7 = 3$, 发现下标 3 刚好为 92, 将该存储单元的数据清空。

0	700
1	50
2	85
3	
4	73
5	101
6	76

平方探测法

所谓 Quadratic Probing，就是每次向下探测的宽度变成了 i 的平方，其中的 i 表示迭代次数。

设 $\text{Hash}(\text{key})$ 表示关键字 key 的哈希值， M 表示哈希表的槽位数（哈希表的大小）。

平方探测法则可以表示为：

如果 $\text{Hash}(x) \% M$ 已经有数据，则尝试 $(\text{Hash}(x) + 1 * 1) \% M$ ；

如果 $(\text{Hash}(x) + 1 * 1) \% M$ 也有数据了，则尝试 $(\text{Hash}(x) + 2 * 2) \% M$ ；

如果 $(\text{Hash}(x) + 2 * 2) \% M$ 也有数据了，则尝试 $(\text{Hash}(x) + 3 * 3) \% M$ ；

双重哈希

对于双重哈希，我们需要一个新的哈希函数 $\text{Hash2}(\text{key})$ ，而且对于第 i 次迭代探测，我们查找的位置为 $i * \text{Hash2}(\text{key})$ 。

设 $\text{Hash}(\text{key})$ 表示关键字 key 的一个哈希值， $\text{Hash2}(\text{key})$ 表示关键字 key 的另一个哈希值， M 表示哈希表的槽位数（哈希表的大小）。

双重哈希探测法则可以表示为：

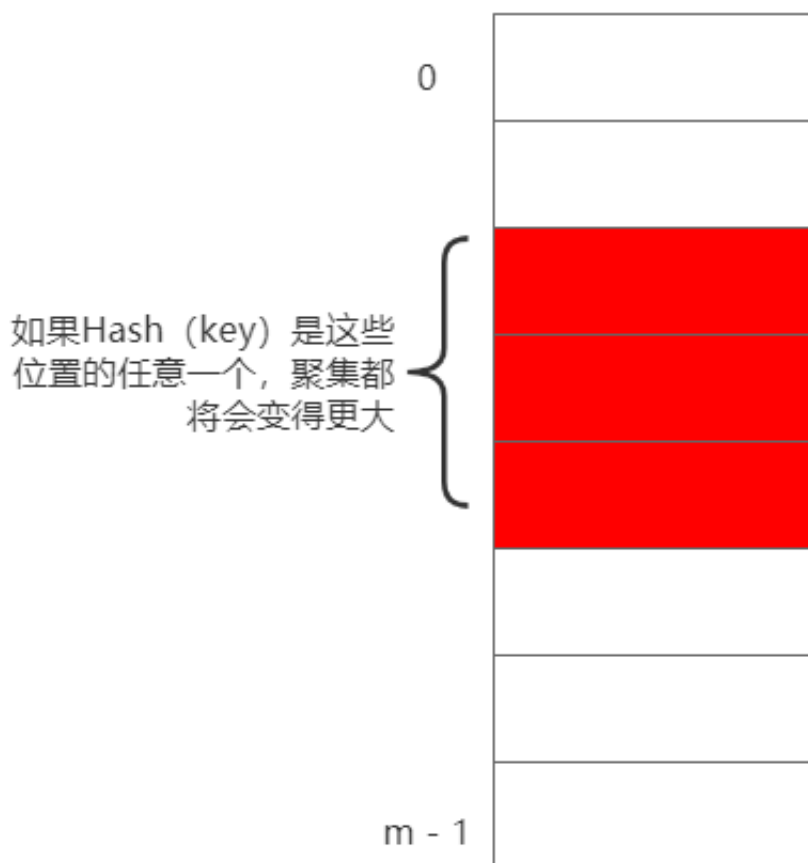
如果 $\text{Hash}(x) \% M$ 已经有数据，则尝试 $(\text{Hash}(x) + 1 * \text{Hash2}(x)) \% M$ ；

如果 $(\text{Hash}(x) + 1 * \text{Hash2}(x)) \% M$ 也有数据了，则尝试 $(\text{Hash}(x) + 2 * \text{Hash2}(x)) \% M$ ；

如果 $(\text{Hash}(x) + 2 * \text{Hash2}(x)) \% M$ 也有数据了，则尝试 $(\text{Hash}(x) + 3 * \text{Hash2}(x)) \% M$ ；

三种探测方法的比较

线性探测具有最佳的缓存性能，但会受到原始聚集（primary cluster）的影响。线性探测易于计算。何为原始聚集（原始聚集是相对于线性探测而言的）？

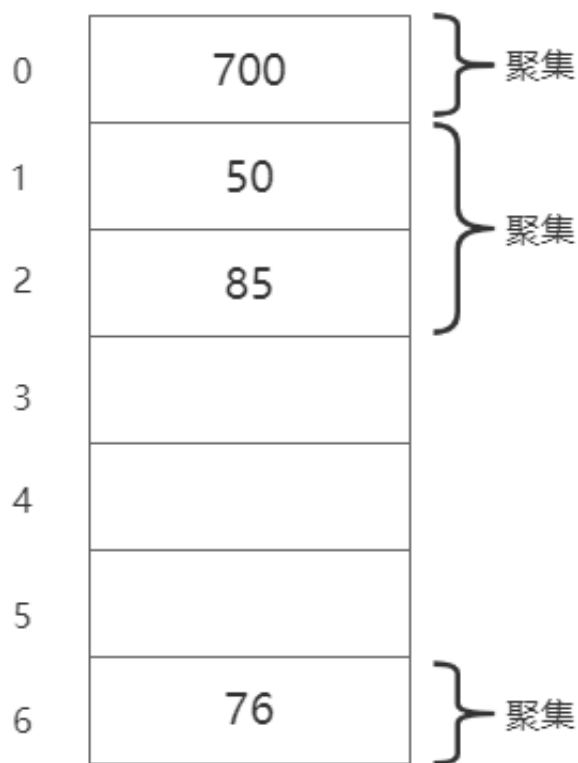


如图所示，如果对于任意一个关键字 `key`，其哈希值 `Hash(key)` 指向图中从左向右的红色箭头的任意位置，聚集（图中的浅蓝色区域）都将得到扩充，而随着聚集的不断扩充，这个聚集会越来越大。

还不理解聚集的概念的话，我们一起看个例子：

0	700	}	聚集
1	50		
2		}	聚集
3			
4			
5		}	聚集
6	76		

上图中的 **700**，**50**，**76** 在插入的时候均未产生碰撞，所以以元素本身单位构成聚集（聚集的大小为 1，此时可以认为不是聚集）。但是之后插入的元素就不一样了，**85** 与 **50** 发生碰撞，插入到了 50 的后面，导致聚集变大。



这就是原始聚集的概念，显然聚集越大，产生碰撞的可能越来越大，哈希算法的性能也会受到影响。

平方探测在缓存性能和受聚集影响方面介于两者中间，平方探测随解决了线性探测的原始聚集问题，但是同时也会产生更细的二次聚集问题。

二重探测的缓存性能较差，但不用担心聚集的情况，但同时由于要计算两个散列函数，时间性能方面受限。

开放地址法和链地址法的比较

所谓比较，算是对前面的一个总结：

No	链地址法	开放地址法
1	易于实现	需要更多的计算
2	使用链地址法，哈希表永远不会填满，不用担心溢出。	哈希表可能被填满，需要通过拷贝来动态扩容。
3	对于哈希函数和装载因子不敏感	需要额外关注如何规避聚集以及装载因子的选择。
4	适合不知道插入和删除的关键字的数量和频率的情况	适合插入和删除的關鍵字已知的情況。
5	由于使用链表来存储关键字，缓存性能差。	所有关键字均存储在同一个哈希表中，所以可以提供更好的缓存性能。
6	空间浪费（哈希表中的有些链一直未被使用）	哈希表中的所有槽位都会被充分利用。
7	指向下一个结点的指针要消耗额外的空间。	不存储指针。

再哈希

对于开放地址法而言，插入、查找和删除的时间复杂度为 $O(1/(1-\alpha))$ ，意味着哈希算法的时间复杂度取决于装填因子 α ， α 越大，开放地址法的时间复杂度也就越高。装填因子 α = 表中填入的记录数字/哈希表的长度

一般将装填因子的值设置为 0.75，随着填入哈希表中记录数的增加，装填因子就会增大甚至超过设置的默认值 0.75，意味着哈希算法的时间复杂度的增加。为了解决装填因子超过默认设置的值 0.75，可以对数组（哈希表）进行扩容（二倍扩容），并将原哈希表中的值进行 **再哈希**，存储到二倍大小的新数组（哈希表）中，从而保证装填因子维持在一个较低的值（不超过 0.75），哈希算法的时间复杂度保证在 $O(1)$ 量级。

这就是为什么需要在哈希的原因所在，下面我们一起看一下再哈希的实现。

再哈希可以大致分为四个步骤：

- 1、对于每一次向哈希表中添加新的键值对，检查装填因子 α 的大小；
- 2、如果 $\alpha > 0.75$ ，则进行 **再哈希**。
- 3、对于再哈希而言，创建一个新的数组（原数组的两倍大小）作为哈希表。
- 4、遍历原数组中的每一个元素，并将其插入到新的数组当中。

我们依旧以 `[50, 700, 76, 85, 92, 73, 101]` 为例进行说明，其中可能会省略部分计算步骤。

开辟一个大小为7的空数组：

--	--	--	--	--	--	--

接下来就是插入并检查装填因子 α ，我们将装填因子自己写到了图中：

	50					
--	----	--	--	--	--	--

装填因子: $\alpha = 1/7 < 3/4$

700	50					
-----	----	--	--	--	--	--

装填因子: $\alpha = 2/7 < 3/4$

700	50					76
-----	----	--	--	--	--	----

装填因子: $\alpha = 3/7 < 3/4$

700	50	85				76
-----	----	----	--	--	--	----

装填因子: $\alpha = 4/7 < 3/4$

700	50	85	92			76
-----	----	----	----	--	--	----

装填因子: $\alpha = 5/7 < 3/4$

700	50	85	92	73		76
-----	----	----	----	----	--	----

装填因子: $\alpha = 6/7 > 3/4$

此时要注意啦，此时装填因子 $\alpha = 6/7 > 3/4$ ，创建一个长度为 14 的新数组（原始数组的两倍）：此时要注意啦，此时装填因子 ，创建一个长度为 14 的新数组（原始数组的两倍）

--	--	--	--	--	--	--	--	--	--	--	--	--	--

此时的数组长度为 14，我们的哈希函数也变了，由原来的 $\text{Hash}(\text{key}) = \text{key} \% 7$ 变成了 $\text{Hash}(\text{key}) = \text{key} \% 14$ 。然后遍历原始数组，并将原始数组中的元素映射到新的数组当中：

700	84		73			76		50	92				
-----	----	--	----	--	--	----	--	----	----	--	--	--	--

然后计算 $101 \% 14 = 3$ ，插入到 **73** 的后面：

700	84		73	101		76		50	92				
-----	----	--	----	-----	--	----	--	----	----	--	--	--	--

这就是再哈希，我相信你也理解啦！但是我们的实现可能不是这样的，我们将插入的是一个键值对，比如 `[<50, "I">, <700, "Love">, <76, "Data">, <85, "Structure">, <92, "and">, <73, "Jing">, <101, "Yu">]`。

哈希算法的应用

哈希算法在日常生活中的应用非常普遍，包括信息摘要（Message Digest），密码校验（Password Verification），数据结构（编程语言），编译操作（Compiler Operation），Rabin-Karp 算法（模式匹配算法），负载均衡等等。

当你注册一个网站在客户端输入邮箱和密码时，客户端会对用户输入的密码进行 hash 运算，然后在服务端的数据库中保存用户密码的 Hash 值，从而保护用户的数据。

C++ 当中的 `unordered_set` & `unordered_map`，以及 Java 中 `HashSet` & `HashMap` 和 Python 中的 `dict` 等各种编程语言均实现了哈希表数据结构。

Rabin-Karp 算法利用哈希来查找字符串的任意一组模式，并且可以用来检查抄袭。

可以利用一致性哈希解决负载均衡问题。

同样可以用于版本校验、防篡改、密码校验，此外还广泛应用于各类数据库当中（包括 MySQL、Redis 等等）。